Hypo-Stage: Tool-Supported Architectural Hypothesis Engineering in Professional Software Development Teams — A Case Study

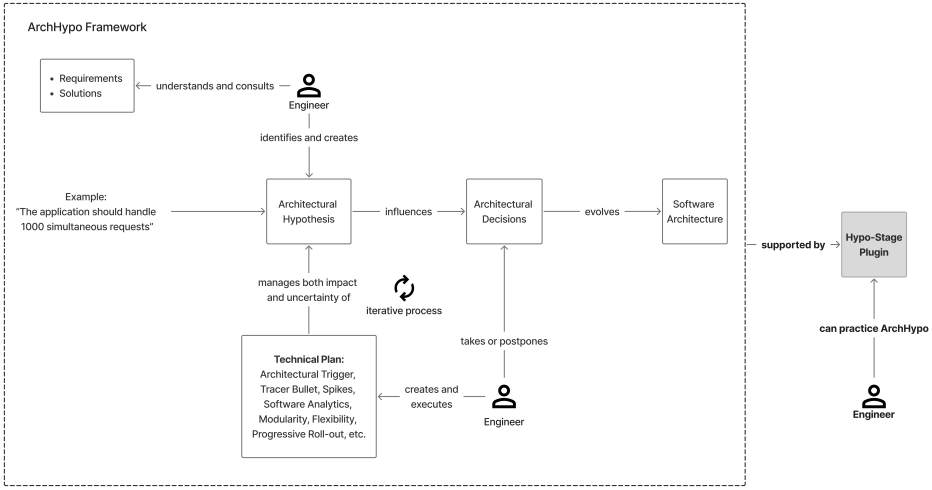
José Gonçalves Lima Neto

11 June 2026

Master's Thesis Defense

Advisor: Prof. Dr. Paulo Roberto Miranda Meirelles
Graduate Program in Computer Science
IME – University of São Paulo

Context


ArchHypo Framework
• Requirements
• Solutions
understands and consults
Engineer
identifies and creates
Example:
"The application should handle 1000 simultaneous requests"
Architectural Hypothesis
influences
Architectural Decisions
evolves
Software Architecture
supported by
Hypo-Stage Plugin
can practice ArchHypo
Engineer
manages both impact and uncertainty of
iterative process
takes or postpones
Technical Plan:
Architectural Trigger, Tracer Bullet, Spikes, Software Analytics, Modularity, Flexibility, Progressive Roll-out, etc.
creates and executes
Engineer

Problem & research questions

The Problem

- **Deriving architectures still depends on architects' tacit knowledge and lacks tool support (Souza et al., 2019).**
- **ArchHypo structures uncertainty, but teams perceive it as hard to learn and missed guidance via supporting tools (Silva, Melegati, Silveira, et al., 2025; Silva, Melegati, Wang, et al., 2024).**
- **Hypo-Stage gives the initial tool support, but impact on real teams and refinements remained unknown (Corrêa, 2025).**

Research questions

RQ1. How do teams *use* Hypo-Stage and ArchHypo on a scaled product?

RQ2. What *uncertainties* and recurring *patterns* do they record?

RQ3. What *benefits and challenges* do team members perceive?

RQ4. Which *refinements* to the tool emerge, and how do they address the challenges?

Study setup

Environment

Single organization | Scaled product | Backstage IDP

Teams

Product team
12 engineers

Platform team
4 engineers

Timeline
5 sprints
(≈9 weeks)

Sprint 1 → Sprint 2 → Sprint 3 → Sprint 4 → Sprint 5

Data sources
triangulated

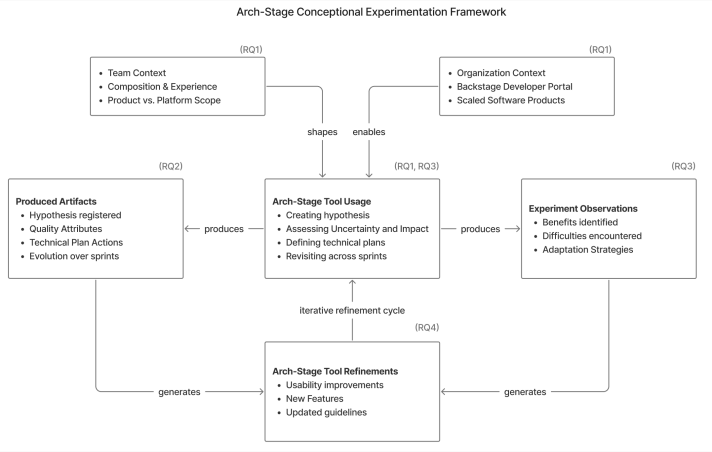
Artifact analysis
(13 hypotheses)

Participant observation

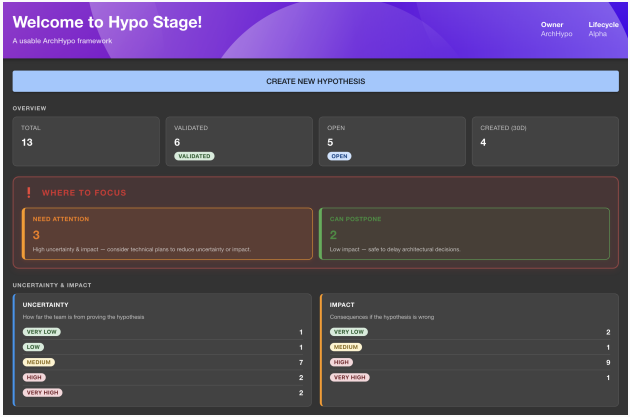
Active-listening sessions

Research Method

Grounded in the flexible design approach to real-world research (Robson and McCartan, 2016).



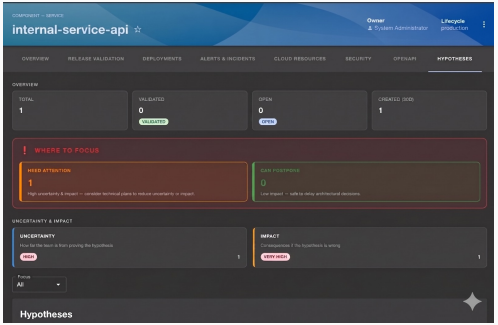
How teams used Hypo-Stage



13 hypotheses, two teams — adoption was senior-led and event-anchored:

- **12 of 13 authored by senior/staff engineers**
- **Seniors used it for decision support; junior/mid uncertainties resolved within a sprint, so went unregistered**
- **Registration clustered around protected technical sessions**
- **Backstage integration cut context-switching cost**

Architectural uncertainties & patterns



Uncertainty × impact, surfaced per service in the developer portal. Dominant quality attributes: **Reliability** and **Auditability**.

Five emerging hypothesis patterns

- **Hypothesis Cluster — one platform concern → linked hypotheses**
- **Regulatory Trigger — compliance obligation as tracked uncertainty**
- **Context-Agnostic Restructuring — whether/how to decouple boundaries**
- **Load-Validated — proven under load, implementation still open**
- **Shared Resource Governance — ownership rules for shared infrastructure**

First empirically grounded answer to *what shapes* architectural hypotheses take in practice.

Perceived benefits & challenges

RQ3

Benefits

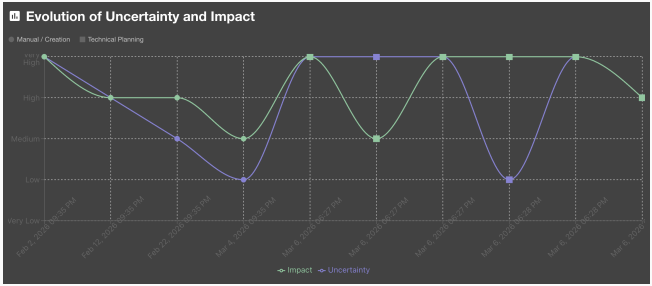
- Externalizes tacit knowledge — a written hypothesis becomes shareable beyond the original discussion
- Supports prioritization — "Can Postpone" justifies deliberate deferral, not neglect
- Collective memory — records open questions before a decision, where ADRs cannot

Challenges

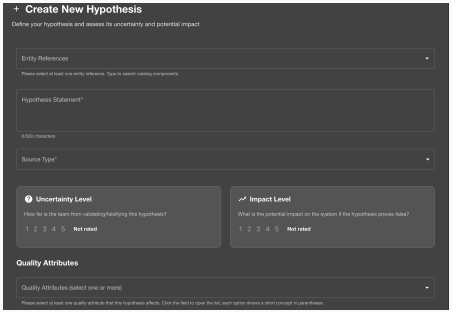
- Falsifiable phrasing — the negative formulation is non-intuitive and easy to skip under pressure
- Registration time cost — short-lived uncertainties go unregistered
- No team ceremonies — usage stayed individual, never a shared review ritual

Tool refinements

RQ4



Evolution chart: uncertainty & impact tracked across successive assessments.



Inline formulation guidance in the creation form lowers the falsifiability barrier.

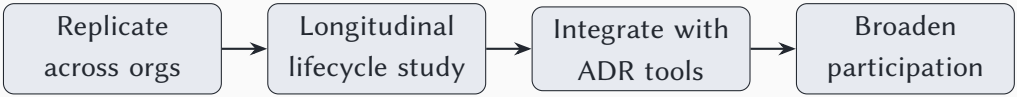
9 refinement categories — including prioritization, the evolution chart, and form usability — each traced to an observed friction point.

Implications & lessons learned

Finding	Implication	Lesson
Even with tool support, hypothesis work stayed senior-anchored (12 of 13).	A tool removes friction but cannot supply the organizational conditions for broad participation.	Ship hypothesis support inside the developer portal — *extend* the workflow, don't add a new one.

Distributed architectural work needs lightweight structure that preserves team autonomy while protecting shared decision quality.

Future work & closing



Takeaway

Tool-mediated ArchHypo succeeds at integration and traceability; the remaining challenge is organizational and pedagogical.

Limitations: single organization, 13 hypotheses, ≈9 weeks, researcher as participant — emerging propositions, not consolidated findings.